

# **DevOps Engineer Challenge - Project Report**

## **End-to-End DevOps: Deploying a React Todo App on AWS with Nginx, PM2, Jenkins, Docker, and CI/CD**

### **1. INTRODUCTION :**

This document provides a detailed walkthrough of the **DevOps Engineer Challenge**, covering the setup, execution, and deployment of a React-based Todo application on an AWS EC2 instance. The challenge involved user management, application deployment using Nginx and PM2, CI/CD pipeline integration with Jenkins, security enhancements, and Dockerization of the application.

### **2. OBJECTIVES :**

The primary objectives of this challenge were:

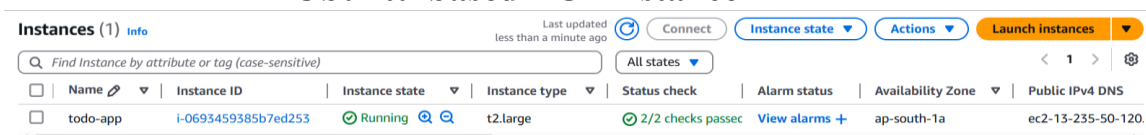
- Setting up an AWS EC2 instance and configuring a secure environment.
- Deploying a React Todo application with Nginx and PM2.
- Configuring a CI/CD pipeline using Jenkins for automated deployment.
- Enhancing security using UFW (Uncomplicated Firewall).
- Dockerizing the application and managing it using Docker Compose.

### **3. ENVIRONMENT SETUP :**

- Operating System: Ubuntu Linux
- Tools & Technologies: Node.js, npm, PM2, Nginx, Jenkins, AWS S3, Docker, Docker Compose
- Cloud Provider: AWS (EC2 instance)

#### **3.1 AWS EC2 Instance Configuration**

1. Launched an **Ubuntu-based EC2 instance** in AWS.



The screenshot displays the AWS Management Console's EC2 Instances page. At the top, there's a search bar with the text 'Find Instance by attribute or tag (case-sensitive)'. To the right of the search bar are buttons for 'Connect', 'Instance state', 'Actions', and 'Launch instances'. Below these is a table with columns: Name, Instance ID, Instance state, Instance type, Status check, Alarm status, Availability Zone, and Public IPv4 DNS. A single instance is listed with the name 'todo-app', ID 'i-0693459385b7ed253', state 'Running', type 't2.large', and status '2/2 checks passed'. The availability zone is 'ap-south-1a' and the public IPv4 DNS is 'ec2-13-235-50-120'.

| Name     | Instance ID         | Instance state | Instance type | Status check      | Alarm status  | Availability Zone | Public IPv4 DNS   |
|----------|---------------------|----------------|---------------|-------------------|---------------|-------------------|-------------------|
| todo-app | i-0693459385b7ed253 | Running        | t2.large      | 2/2 checks passed | View alarms + | ap-south-1a       | ec2-13-235-50-120 |

2. Connected to the instance using SSH:

```
ssh -i your-key.pem ubuntu@<EC2-Public-IP>
```

3. Updated the system:

```
sudo apt update && apt upgrade -y
```

### **3.2 User Management**

1. Created a new user named DevOps:

```
sudo adduser -m DevOps
```

2. Granted sudo privileges to the user:

```
sudo usermod -aG sudo DevOps
```

3. Remove Password authentication:

```
sudo passwd -d DevOps
```

4. Switch to DevOps User:

```
sudo su - DevOps
```

```
DevOps@ip-172-31-38-158:~$ whoami
DevOps
DevOps@ip-172-31-38-158:~$ groups DevOps
DevOps : DevOps sudo
DevOps@ip-172-31-38-158:~$
```

---

## **4. REACT APPLICATION DEPLOYMENT :**

### **4.1 Install Required Software**

1. Installed Node.js and npm:

```
sudo apt install nodejs npm -y
```

2. Verified installation:

```
node --version
```

```
npm --version
```

## 4.2 Clone & Build the React Project

1. Cloned the repository:

```
sudo git clone https://github.com/kabirbaidhya/react-todo-app.git /opt/checkout/react-todo-app
```

2. Installed project dependencies:

```
cd /opt/checkout/react-todo-add
```

```
sudo npm install
```

3. Built the project:

```
sudo npm run build
```

4. Created a deployment directory and moved the build files:

```
sudo mv build react
```

```
sudo mkdir -p /opt/deployment
```

```
sudo cp -r react /opt/deployment
```

## 4.3 Deploy Using PM2

1. Installed PM2:

```
sudo npm install -g serve pm2
```

2. Started serving the React app:

```
pm2 start npx --name react-app -- serve -s  
/opt/deployment/react
```

3. Configured PM2 to start on boot:

```
pm2 save --force
```

```
pm2 startup
```

```
PM2: Spawning PM2 daemon with pm2_home=/home/DevOps/.pm2
PM2: PM2 Successfully daemonized
PM2: Starting /usr/bin/npx in fork_mode (1 instance)
PM2: Done.



| id | name      | namespace | version | mode | pid   | uptime | 0 | status | cpu | mem    | user   | watching |
|----|-----------|-----------|---------|------|-------|--------|---|--------|-----|--------|--------|----------|
| 0  | react-app | default   | N/A     | fork | 25784 | 0s     | 0 | online | 0%  | 27.3mb | DevOps | disabled |



DevOps@ip-172-31-38-158:/opt/checkout/react-todo-app$ pm2 save --force
PM2: Saving current process list...
PM2: Successfully saved in /home/DevOps/.pm2/dump.pm2
DevOps@ip-172-31-38-158:/opt/checkout/react-todo-app$ pm2 startup
PM2: Init System found: systemd
PM2: To setup the Startup Script, copy/paste the following command:
sudo env PATH=$PATH:/usr/bin:/usr/local/lib/node_modules/pm2/bin/pm2 startup systemd -u DevOps --hp /home/DevOps
DevOps@ip-172-31-38-158:/opt/checkout/react-todo-app$
```

## **5. NGINX CONFIGURATION & SECURITY :**

### **5.1 Install & Configure Nginx**

1. Installed Nginx:

```
sudo apt install nginx -y
```

2. Created a new Nginx configuration file:

```
sudo nano /etc/nginx/sites-available/default
```

3. Added the following content:

```
server {  
    listen 80;  
    location / {  
        proxy_pass http://localhost:3000;  
        proxy_http_version 1.1;  
        proxy_set_header Upgrade $http_upgrade;  
        proxy_set_header Connection 'upgrade';  
        proxy_set_header Host $host;  
        proxy_cache_bypass $http_upgrade;  
    }  
}
```

4. Enabled the configuration and restarted Nginx:

```
sudo nginx -t
```

```
sudo systemctl restart nginx
```

### **5.2 Security Enhancements**

1. Configured UFW firewall to allow only necessary ports:

```
sudo ufw allow 80
```

```
sudo ufw allow 443
```

```
sudo ufw allow 3000
```

```
sudo ufw allow 22
```

```
sudo ufw enable
```

2. Verified firewall settings:

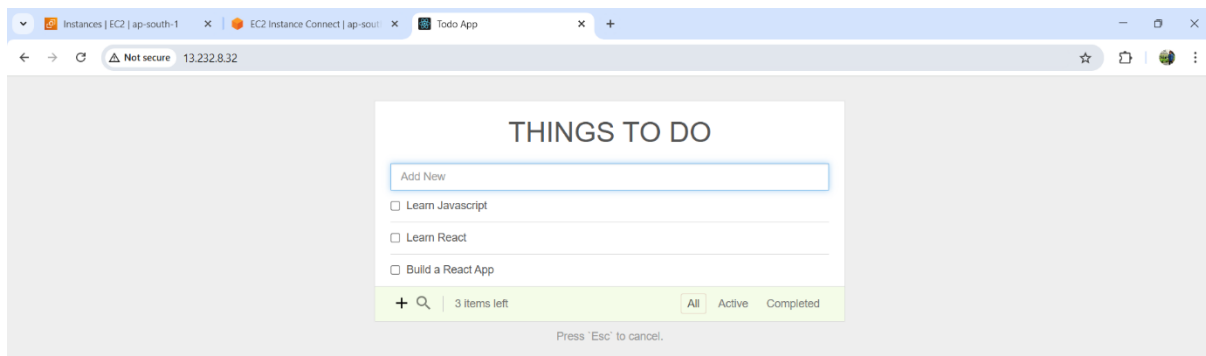
```
sudo ufw status
```

### 5.3: Verify the Deployment

- Open a browser and visit:

***http://<EC2-Public-IP>***

- You should see the React Todo App:



## **6. CI/CD PIPELINE WITH JENKINS :**

### **6.1 Install Jenkins**

1. Installed Java (required for Jenkins):

```
sudo apt install fontconfig openjdk-17-jre
```

2. Installed Jenkins:

```
sudo wget -O /usr/share/keyrings/jenkins-keyring.asc \
```

```
https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key
```

```
echo "deb [signed-by=/usr/share/keyrings/jenkins-  
keyring.asc]" \
```

```
https://pkg.jenkins.io/debian-stable binary/ | sudo tee \  
/etc/apt/sources.list.d/jenkins.list > /dev/null
```

```
sudo apt-get update
```

```
sudo apt-get install jenkins
```

3. Add Jenkins to the sudo group:

```
sudo usermod -aG sudo jenkins
```

4. Allow Jenkins to use sudo without password:

```
sudo visudo
```

Add this line:

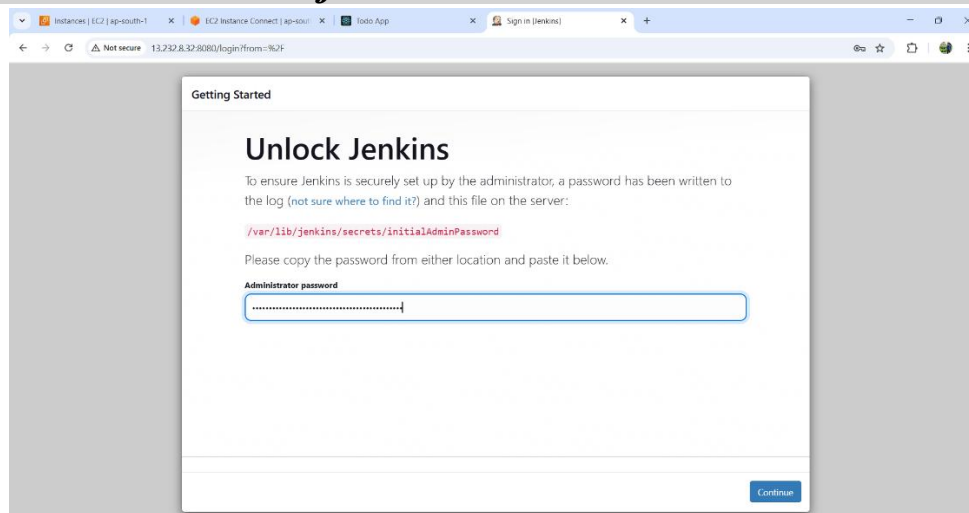
```
jenkins ALL=(ALL) NOPASSWD: ALL
```

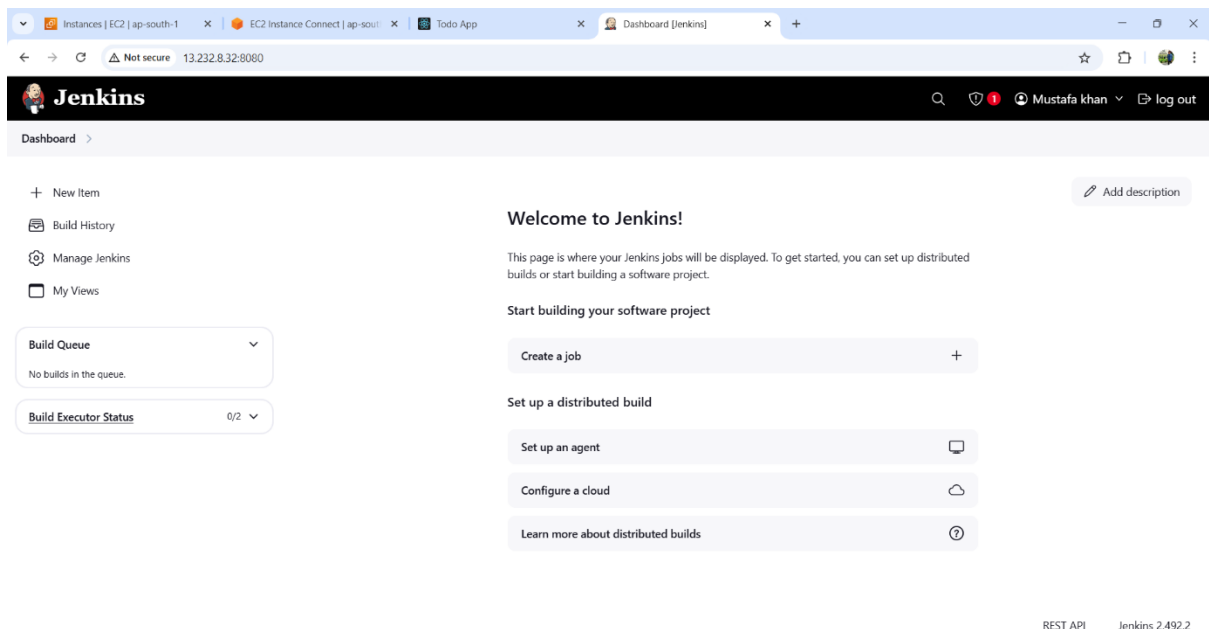
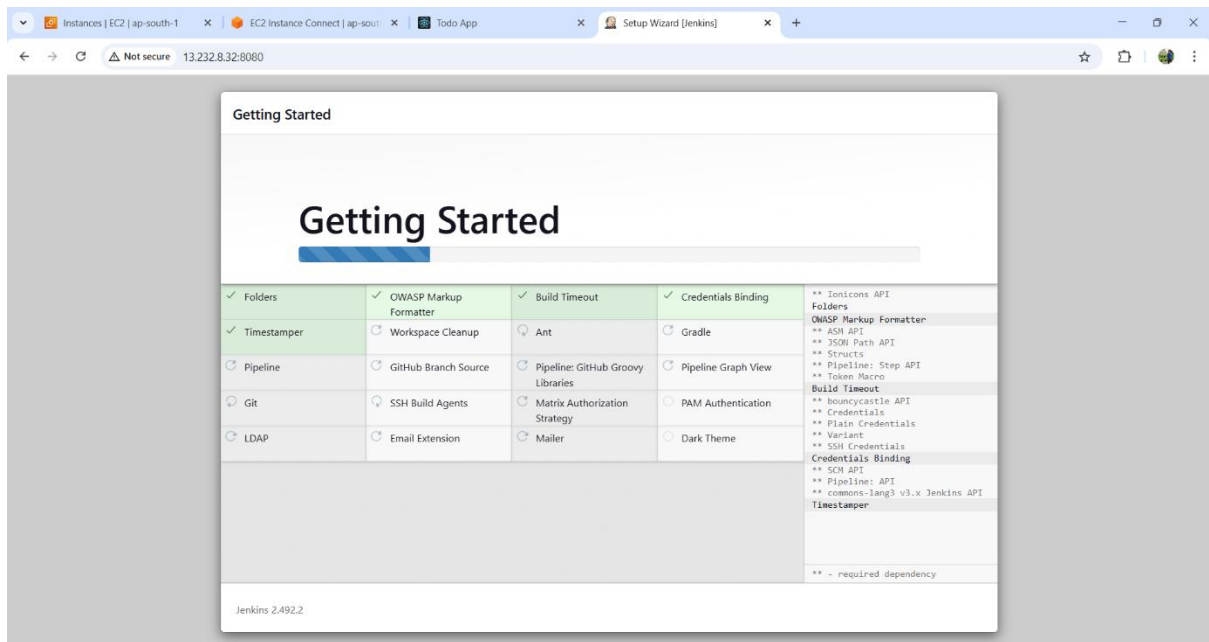
5. Open a browser and access Jenkins:

```
http://<EC2-Public-IP>:8080
```

6. Retrieved the Jenkins admin password:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```





## 6.2 Install Plugins

After initial setup:

1. Go to Manage Jenkins > Plugins > Available Plugins
2. Search for and install:
  - Pipeline: AWS Steps
  - AWS Credentials Plugin

## 6.3 Configure AWS Credentials in Jenkins

1. Go to Manage Jenkins > Credentials > Global > Add Credentials

2. Select AWS Credentials
3. Enter:
  - ID: aws-credentials-id
  - Access Key ID: (from AWS)
  - Secret Access Key: (from AWS)

#### 6.4 Create a Jenkins Pipeline Job

1. Go to **Jenkins Dashboard** → **New Item**.
2. Select **Pipeline**, name it React-Todo-CICD, and click **OK**.
3. Scroll down to the **Pipeline** section and select **Pipeline Script**.
4. Add the following pipeline script:

```
pipeline {  
    agent any  
    environment {  
        APP_NAME = 'react-app'  
        GIT_REPO = 'https://github.com/kabirbaidhya/react-  
todo-app.git'  
        CHECKOUT_DIR = '/opt/checkout/react-todo-add'  
        BUILD_DIR = '/opt/deployment/react'  
        S3_BUCKET    =    'react-tod-app-s3bucket-devops-  
challenge'  
    }  
    stages {  
        stage('Stop Existing Deployment') {  
            steps {  
                sh 'sudo -u DevOps pm2 stop all || echo "App not  
running"'
```



```

    }
}

stage('Pull Fresh Code') {
    steps {
        sh '''
            sudo chown -R jenkins:jenkins /opt/checkout/
/opt/deployment/

            sudo rm -rf $CHECKOUT_DIR || true
            sudo git clone $GIT_REPO $CHECKOUT_DIR
            '''
        }
    }

stage('Build Application') {
    steps {
        dir("${CHECKOUT_DIR}") {
            sh 'sudo npm install'
            sh 'sudo npm run build'
        }
        sh '''
            sudo rm -rf $BUILD_DIR || true
            sudo mkdir -p $BUILD_DIR
            sudo cp -r $CHECKOUT_DIR/build/*
$BUILD_DIR/

```

```

'''
}
}
stage('Deploy with PM2') {
  steps {
    sh '''
      sudo pm2 start npx --name $APP_NAME -- serve -
s $BUILD_DIR
      sudo pm2 save --force
      sudo pm2 startup
    '''
  }
}
stage('Upload to S3') {
  steps {
    withAWS(credentials: '390402563998') {
      s3Upload(
        bucket: "$S3_BUCKET",
        file: "$BUILD_DIR",
      )
    }
  }
}
}

```

```

}

post {

    success { echo 'Deployment and S3 upload successful!' }

    failure { echo 'Deployment or S3 upload failed.' }

}

}

```

## 6.5 Run and Monitor the Pipeline

- Click Build Now to start the pipeline.
- View logs under Build Console Output.

The screenshot shows the Jenkins dashboard for the 'React-App' pipeline. The 'Stage View' section displays a table of stage execution times for three builds. The 'Builds' section on the left shows a list of builds with their status and timestamps. The 'Permalinks' section provides links to the last build, last stable build, and last successful build.

| Stage                                      | Stop Existing Deployment | Pull Fresh Code | Build Application | Deploy with PM2 | Upload to S3 | Declarative: Post Actions |
|--------------------------------------------|--------------------------|-----------------|-------------------|-----------------|--------------|---------------------------|
| Average stage times: (full run time: ~49s) | 741ms                    | 1s              | 33s               | 2s              | 1s           | 116ms                     |
| #3 01:52 No Changes                        | 616ms                    | 1s              | 16s               |                 |              |                           |
| #2 01:47 No Changes                        | 626ms                    | 2s              | 24s               | 1s              | 488ms        | 119ms                     |
| #1 01:46 No Changes                        | 982ms                    | 1s              | 58s               | 2s              | 1s           | 113ms                     |

**Builds**

Today

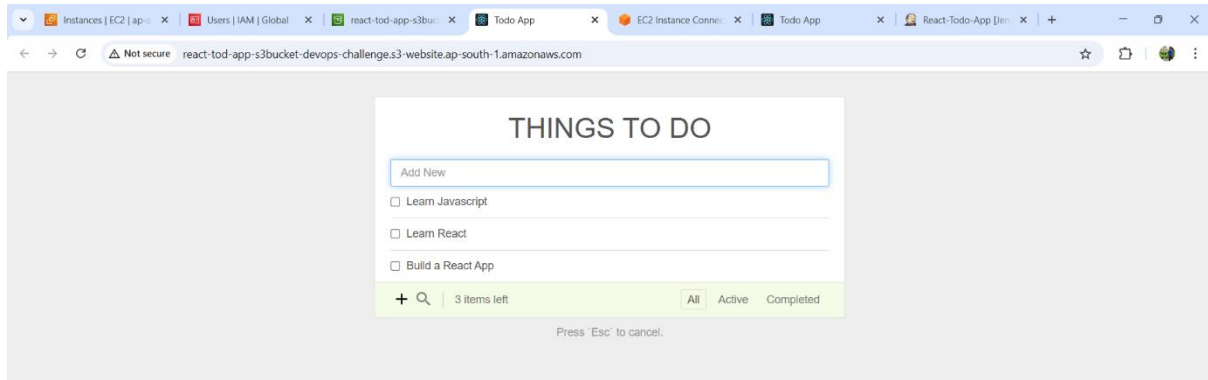
- #3 20:22
- #2 20:17
- #1 20:16

**Permalinks**

- Last build (#3), 8.5 sec ago
- Last stable build (#2), 4 min 36 sec ago
- Last successful build (#2), 4 min 36 sec ago

The screenshot shows the Amazon S3 console for the bucket 'react-tod-app-s3bucket-devops-challenge'. The 'Objects' tab is selected, showing a list of objects with their names, types, last modified dates, sizes, and storage classes.

| Name                | Type   | Last modified                        | Size    | Storage class |
|---------------------|--------|--------------------------------------|---------|---------------|
| asset-manifest.json | json   | March 21, 2025, 01:48:16 (UTC+05:30) | 871.0 B | Standard      |
| favicon.ico         | ico    | March 21, 2025, 01:48:16 (UTC+05:30) | 24.3 KB | Standard      |
| index.html          | html   | March 21, 2025, 01:48:16 (UTC+05:30) | 377.0 B | Standard      |
| static/             | Folder | -                                    | -       | -             |



## 7. SHELL SCRIPTING FOR JAVA INSTALLATION :

1. Created a Bash script for installing OpenJDK 1.8:

```
sudo nano /opt/scripts/install_java.sh
```

2. Added the following content:

```
#!/bin/bash  
  
echo "$(date) - Starting Java Installation" |  
tee -a /opt/logs/script_logs.log  
  
sudo apt update && sudo apt install -y openjdk-8-jdk  
  
echo "$(date) - Java Installed" | tee -a /opt/logs/script_logs.log
```

3. Made the script executable and ran it:

```
chmod +x /opt/scripts/install_java.sh  
  
sudo /opt/scripts/install_java.sh
```

## 8. DOCKERIZATION OF THE REACT APPLICATION :

### 8.1 Install Docker & Docker Compose

```
sudo apt install docker.io -y  
  
sudo usermod -aG docker $USER  
  
sudo reboot
```

## 8.2 Install Docker Compose:

```
mkdir -p ~/.docker/cli-plugins/
```

```
curl -SL https://github.com/docker/compose/releases/download/v2.3.3/docker-  
compose-linux-x86_64 -o ~/.docker/cli-plugins/docker-compose
```

```
chmod +x ~/.docker/cli-plugins/docker-compose
```

## 8.3 Clone the repository and configure Nginx:

```
git clone https://github.com/kabirbaidhya/react-todo-  
app.git
```

```
cd react-todo-app
```

```
mkdir .docker && cd .docker
```

## 8.4 Create and configure nginx.conf:

```
events { worker_connections 1024; }  
http {  
    include /etc/nginx/mime.types;  
    server {  
        listen 80;  
        location / {  
            root /usr/share/nginx/html;  
            index index.html;  
            try_files $uri $uri/ /index.html;  
        }  
        error_page 404 /index.html;
```

```
}  
}
```

### 8.5 Create Dockerfile:

***nano Dockerfile***

Add Content:

```
FROM node:18 AS build  
WORKDIR /app  
COPY package*.json ./  
RUN npm install  
COPY . .  
RUN npm run build  
FROM nginx:1.25.1  
COPY .docker/nginx.conf /etc/nginx/nginx.conf  
COPY --from=build /app/build /usr/share/nginx/html  
EXPOSE 80  
CMD ["nginx", "-g", "daemon off;"]
```

### 8.6 Create a Docker Compose File

***sudo nano /opt/deployment/react/docker-compose.yml***

Content:

```
version: '3'  
services:  
  react-app:
```

**build:**

**context: .**

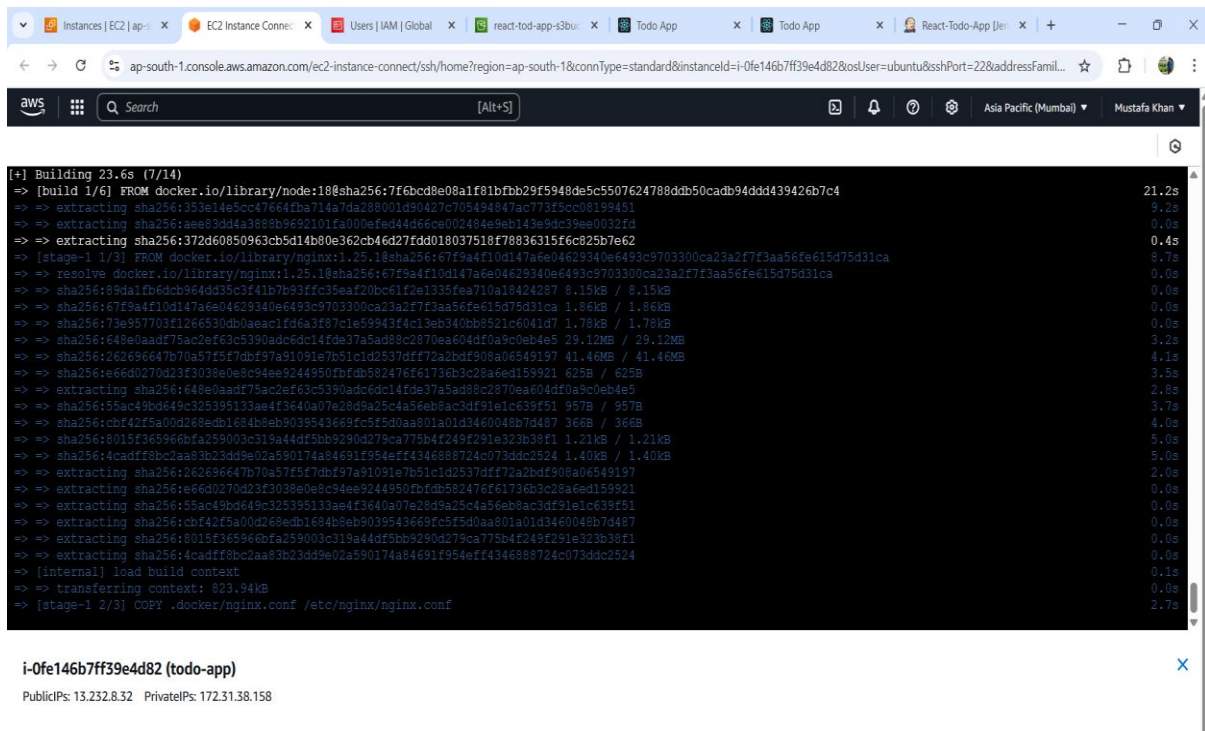
**dockerfile: Dockerfile**

**ports:**

**- "3000:80"**

## 8.7 Run the Container

***docker-compose up -d***



```
[+] Building 23.6s (7/14)
=> [build 1/6] FROM docker.io/library/nginx:1.25.1:sha256:7f6bcd9e08a1f01bfbb29f5948de5c5507624788db50cadb94ddd439426b7c4 21.2s
=> extracting sha256:1353e14e5cc4f664fba714a7da288001d90427c705494847ac773f3cc08199451 9.2s
=> extracting sha256:1aee83d84a3898b9692101fa000e0e44466c002484e9eb143e9a39ee0022f4 0.0s
=> extracting sha256:372d60850963cb5d14b80e362cb46d27fdd018037518f78836315f6c825b7e62 0.4s
=> [stage-1 1/3] FROM docker.io/library/nginx:1.25.1:sha256:7f6bcd9e08a1f01bfbb29f5948de5c5507624788db50cadb94ddd439426b7c4 8.7s
=> resolve docker.io/library/nginx:1.25.1:sha256:7f6bcd9e08a1f01bfbb29f5948de5c5507624788db50cadb94ddd439426b7c4 0.0s
=> sha256:89da1fb6dc964dd35c3f41b7b93ffc39eaf20bc61f2e1335fea710a18424287 8.15KB / 8.15KB 0.0s
=> sha256:67f9a4f10d147a6e04629340e6493c9703300ca23a2f7f3aa56fe615d75d31ca 1.86KB / 1.86KB 0.0s
=> sha256:73e957703f1266530db0aeadcf6da3f87c1e59943f4c13eb340bb8521c6041d7 1.78KB / 1.78KB 0.0s
=> sha256:648e0aadf75ac2ef63c390adc6dc14fde37a5ad88c2870ea604df0a9c0eb4e5 29.12MB / 29.12MB 3.2s
=> sha256:262696647b70a57f5f7dbf97a91091e7b51c1d2537dff72a2bdf908a06549197 41.46MB / 41.46MB 4.1s
=> sha256:e66d0270d23f3038e0e8c94ee9244950fbfcb582476f61736b3c28a6ed159921 625B / 625B 3.5s
=> extracting sha256:648e0aadf75ac2ef63c390adc6dc14fde37a5ad88c2870ea604df0a9c0eb4e5 2.8s
=> sha256:55ac49bd649c325395133ae4f3640a07e28d9a25c4a56eb8ac3df91e1c639f51 957B / 957B 3.7s
=> sha256:cbf42f5a00d268edb1684b8eb9039543669f5f5d0aa801a01d3460048b7d487 366B / 366B 4.0s
=> sha256:8015f365966bfa259003c319a44df5bb9290d279ca775b4f249f291e323b38f1 1.21KB / 1.21KB 5.0s
=> sha256:4cadff8bc2aa83b23dd9e02a590174a84691f954eff4346888724c073ddc2524 1.40KB / 1.40KB 5.0s
=> extracting sha256:262696647b70a57f5f7dbf97a91091e7b51c1d2537dff72a2bdf908a06549197 2.0s
=> extracting sha256:e66d0270d23f3038e0e8c94ee9244950fbfcb582476f61736b3c28a6ed159921 0.0s
=> extracting sha256:55ac49bd649c325395133ae4f3640a07e28d9a25c4a56eb8ac3df91e1c639f51 0.0s
=> extracting sha256:cbf42f5a00d268edb1684b8eb9039543669f5f5d0aa801a01d3460048b7d487 0.0s
=> extracting sha256:8015f365966bfa259003c319a44df5bb9290d279ca775b4f249f291e323b38f1 0.0s
=> extracting sha256:4cadff8bc2aa83b23dd9e02a590174a84691f954eff4346888724c073ddc2524 0.0s
=> [internal] load build context 0.1s
=> transferring context: 823.94KB 0.0s
=> [stage-1 2/3] COPY .docker/nginx.conf /etc/nginx/nginx.conf 2.7s
```

i-0fe146b7ff39e4d82 (todo-app)

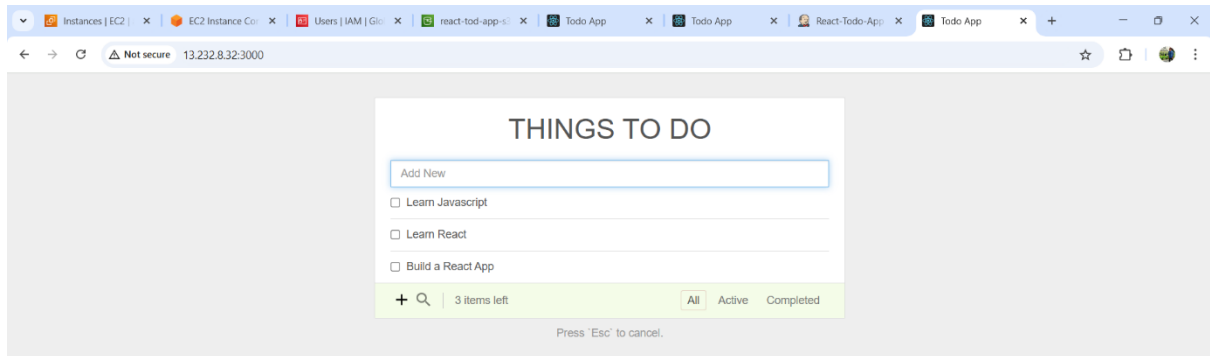
PublicIPs: 13.232.8.32 PrivateIPs: 172.31.38.158

***docker ps***

## 9. EXECUTION & TESTING :

- Access the app via **http://<EC2-Public-IP>**.
- Verify Dockerized version via **http://<EC2-Public-IP>:3000**.
- Check Jenkins pipeline execution logs.

## 10. OUTPUT SCREENSHOT:



---

## 11. CONCLUSION :

This challenge provided hands-on experience in DevOps workflows, cloud deployment, CI/CD automation, and containerization. The project successfully established a **secure, automated deployment pipeline** for the React application.

---

## 12. FUTURE ENHANCEMENTS :

- Integrate Kubernetes for container orchestration.
- Implement monitoring with Prometheus and Grafana.
- Automate infrastructure provisioning using Terraform.